

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 2 | SLO 1: Programming Language Theory (Code: 1021)

Q: 1. Syntax Check: Find and fix the syntax error in this Java code:

```
public class Test {  
    public static void main(String[] args) {  
        int num = 10  
        System.out.println(num);  
    }  
}
```

Ans: Error: Missing statement terminator semicolon (;) after int num = 10 on line 3.

Corrected Code:

```
public class Test {  
    public static void main(String[] args) {  
        int num = 10; // Fixed: added semicolon  
        System.out.println(num);  
    }  
}
```

Q: 2. Semantics Understanding: What will be the output of this code, and why?

```
int x = 5;  
int y = x;  
x = 10;  
System.out.println(y);
```

Ans: Output: 5

Explanation: In Java, primitive data types (like int) are passed and copied entirely by value. When int y = x; executes, the bitwise value of x (which is 5) is copied directly into y's distinct memory location. Modifying x = 10; afterwards has zero effect on y.

Q: 3. Pragmatics Insight: In Java, why is using ArrayList generally preferred over arrays for dynamic collections in modern programming environments?

Ans: Explanation:

- **Dynamic Resizing:** Standard Java arrays have fixed capacity defined at instantiation. ArrayList dynamically grows its internal storage buffer automatically as elements are appended.
- **Rich Utility Methods:** Provides built-in methods like add(), remove(), contains(), indexOf(), and clear().
- **Generics & Type Safety:** Enforces compile-time type checking with generic parameters (e.g., ArrayList<String>).
- **Interoperability:** Integrates seamlessly with the Java Collections Framework, iterators, and Java Stream API for functional filtering and mapping.